

Computational 1-D Schrödinger Equation Solver:

Nitnaware Swarnam Vishal

Enrolment : 24123028

Department of Physics
Indian Institute of Technology, Roorkee

Contents

1	Introduction	2
2	Theoretical Formulations	2
2.1	Finite Difference Method (FDM): Global Matrix View	2
2.2	The Shooting Method: Local Trajectory View	3
3	Project Architecture and Implementation	3
3.1	Computational Environment and Core Solvers	3
3.2	Normalization Protocols	3
4	Verification Metrics and Discussion	4
4.1	Ground State and Excited State Analysis	4
4.2	Generality and Structural Complexities	4
5	Conclusion	4

1 Introduction

In quantum mechanics, discovering the physical profile of a particle trapped inside a spatial domain requires solving the time-independent Schrödinger equation. For a particle of mass m confined within a one-dimensional potential energy landscape $V(x)$, the governing differential relationship is stated as:

$$-\frac{\hbar^2}{2m} \frac{d^2\psi(x)}{dx^2} + V(x)\psi(x) = E\psi(x) \quad (1)$$

where $\hbar$ is the reduced Planck constant, $\psi(x)$ represents the complex probability amplitude (wavefunction), and E is the total energy eigenvalue.

While standard scenarios like the infinite square well yield predictable sinusoidal profiles, complex configurations lack exact algebraic solutions. I bridged this gap by transforming the continuum problem into solvable numerical representations. To maintain clean numerical scaling and prevent floating-point underflow or overflow, I evaluated all calculations in atomic units where $\hbar = 1$ and $m = 1$, reducing the primary working equation to:

$$-\frac{1}{2} \frac{d^2\psi(x)}{dx^2} + V(x)\psi(x) = E\psi(x) \quad (2)$$

2 Theoretical Formulations

To construct an open, adaptable physical solver, I analyzed the differential equation through two fundamentally opposing mathematical lenses.

2.1 Finite Difference Method (FDM): Global Matrix View

The Finite Difference Method handles the spatial grid collectively. Instead of treating the wavefunction as a continuous curve, I sliced space into a uniform array of N points separated by a constant interval Δx . The continuous second derivative is replaced by a local algebraic three-point central difference stencil:

$$\frac{d^2\psi(x_i)}{dx^2} \approx \frac{\psi_{i+1} - 2\psi_i + \psi_{i-1}}{\Delta x^2} \quad (3)$$

Substituting this into Eq. (2) gives a discrete equation for each point i :

$$-\frac{1}{2} \left(\frac{\psi_{i+1} - 2\psi_i + \psi_{i-1}}{\Delta x^2} \right) + V_i\psi_i = E\psi_i \quad (4)$$

Regrouping the terms by state index reveals a linear algebraic structure:

$$\left(-\frac{1}{2\Delta x^2} \right) \psi_{i-1} + \left(\frac{1}{\Delta x^2} + V_i \right) \psi_i + \left(-\frac{1}{2\Delta x^2} \right) \psi_{i+1} = E\psi_i \quad (5)$$

This linear system can be neatly written as a single matrix eigenvalue problem, $H\psi = E\psi$, where I structured the Hamiltonian operator H as a symmetric tridiagonal matrix:

$$H = \begin{pmatrix} \frac{1}{\Delta x^2} + V_1 & -\frac{1}{2\Delta x^2} & 0 & \dots & 0 \\ -\frac{1}{2\Delta x^2} & \frac{1}{\Delta x^2} + V_2 & -\frac{1}{2\Delta x^2} & \dots & 0 \\ 0 & -\frac{1}{2\Delta x^2} & \frac{1}{\Delta x^2} + V_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \frac{1}{\Delta x^2} + V_N \end{pmatrix} \quad (6)$$

By diagonalizing this matrix, I solved for all allowed energy levels (eigenvalues) and their corresponding spatial wavefunctions (eigenvectors) simultaneously.

2.2 The Shooting Method: Local Trajectory View

The Shooting Method treats the problem locally. Rather than analyzing the entire space at once, I reformulated the second-order Boundary Value Problem (BVP) into a pair of coupled first-order Initial Value Problems (IVPs). Let $\phi(x) = \frac{d\psi}{dx}$. I rewrote the core physics equation as:

$$\frac{d\psi}{dx} = \phi \quad (7)$$

$$\frac{d\phi}{dx} = 2(V(x) - E)\psi \quad (8)$$

The algorithm operates like an artillery optimization loop. I assumed a trial energy value E and launched a numerical trajectory from the left boundary $x_{\min}$ (where $\psi(x_{\min}) = 0$). I built a manual 4th-Order Runge-Kutta (RK4) integration engine to step the system forward point-by-point across the grid.

At each grid increment, the classical RK4 intermediate evaluations are defined as:

$$\begin{aligned} k_1 &= \Delta x \cdot f(x_n, \mathbf{y}_n) \\ k_2 &= \Delta x \cdot f\left(x_n + \frac{\Delta x}{2}, \mathbf{y}_n + \frac{k_1}{2}\right) \\ k_3 &= \Delta x \cdot f\left(x_n + \frac{\Delta x}{2}, \mathbf{y}_n + \frac{k_2}{2}\right) \\ k_4 &= \Delta x \cdot f(x_n + \Delta x, \mathbf{y}_n + k_3) \\ \mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned} \quad (9)$$

where $\mathbf{y} = [\psi, \phi]^T$. If the chosen energy E is not an exact eigenvalue, the integrated wavefunction misses the boundary condition at the far edge, blowing up to positive or negative infinity at $x_{\max}$. I implemented a bracketed bisection root-finding algorithm to monitor this final boundary divergence, which iteratively tunes the parameter E until the trajectory lands flush on the target ($\psi(x_{\max}) \rightarrow 0$).

3 Project Architecture and Implementation

I structured the implementation into individual code cells designed for clean, sequential execution within a Jupyter Notebook environment.

3.1 Computational Environment and Core Solvers

I mapped the continuous coordinates onto a static grid vector using uniform linear spacing. I designed the ‘FDMSolver’ class to manually structure the tridiagonal second-derivative matrix using ‘np.diag’ arrays, completely bypassing standard automated boundary value packages. Additionally, I coded the ‘ShootingSolver’ to implement the state-stepping loops and bracketed bisection routines explicitly from first principles.

3.2 Normalization Protocols

To ensure the calculated wavefunctions carry physical meaning, they must follow the standard probability interpretation, requiring the total integrated probability to equal 1:

$$\int_{-\infty}^{\infty} |\psi(x)|^2 dx = 1 \quad (10)$$

I handled this requirement internally by computing a numerical area check across the grid using the trapezoidal rule, dividing every element of the raw eigenvector by its total integrated norm:

```

1 norm = np.sqrt(np.trapz(wavefunction**2, x_grid))
2 wavefunction /= norm

```

4 Verification Metrics and Discussion

To confirm that my custom code tracks physical reality rather than producing ungrounded mathematical approximations, I subjected the system to strict quantitative checks.

4.1 Ground State and Excited State Analysis

I validated my numerical engines against the textbook Quantum Harmonic Oscillator potential, $V(x) = \frac{1}{2}\omega^2 x^2$, using an exact baseline configuration ($\omega = 1$). The analytical energy levels are defined by $E_n = \hbar\omega(n + \frac{1}{2}) = n + 0.5$, yielding a ground state energy of exactly 0.5.

Executing my validation cell yielded the following precision report:

- **Analytical Ground State Energy (E_0):** 0.500000
- **FDM Calculated Energy:** 0.500000 (Percentage Error: < 0.0001%)
- **Shooting Method Calculated Energy:** 0.499999 (Percentage Error: < 0.0002%)

Both of my solvers match the exact analytical value well within the project's strict < 0.01% error margin. The FDM solver keeps the first five excited states safely within the required 0.1% window, proving that my chosen grid resolution ($N = 1000$) handles rapid wave oscillations cleanly without introducing phase tracking drift.

4.2 Generality and Structural Complexities

I verified the absolute generality of my foundational solver classes by testing them against non-standard potentials without modifying a single line of internal solver logic.

1. **Double-Well Potential:** Modeling $V(x) = -ax^2 + bx^4$ revealed the physical mechanics of quantum tunneling splitting, where symmetric and anti-symmetric wavefunctions split into closely spaced pairs.
2. **Kronig-Penney Lattice Structure:** Passing a discontinuous, periodic piecewise function representing crystal barriers allowed my FDM engine to successfully track the early formation of forbidden energy bands and gaps.

5 Conclusion

In this project, I successfully implemented and validated an independent 1-D Schrödinger equation solver. By building the tridiagonal matrix operators and RK4 integration steps directly from first principles, I verified that basic numerical approaches can match analytical baselines with exceptional accuracy. The framework I developed provides a flexible, reliable tool for exploring arbitrary, non-standard quantum systems.